

RHEINWERK AGENT WORKSHOP

HANDS-ON CHALLENGE 1: SALES SUPPORT AGENT

(Version: September 2026, Joule Studio, classic edition)

0. Worum es geht

Die **Norbach Antriebstechnik GmbH** ist ein fiktiver Mittelständler, der Antriebs- und Messkomponenten für den Anlagenbau baut. Ihre Key Account Manager bereiten Kundentermine vor, indem sie CRM, Projektablage, Mailverläufe und ERP-Export einzeln durchgehen.

Euer Ziel: Ein Agent, der diesen Kontext auf Zuruf zusammenzieht und die relevanten Kennzahlen ausrechnet. Er liest, rechnet und fasst zusammen. Er verschickt nichts, schreibt keine Mails und macht keine Preiszusagen.

Ihr baut den Agenten zuerst mit einer absichtlich unfertigen Konfiguration. Er wird Fehler machen. Diese Fehler zu finden ist der Inhalt der Session, das Zusammenklicken der Tools ist nur die Vorarbeit dazu.

1. Setup

1.1 Öffnet das GitHub-Repository der Challenge: <https://github.com/daniel-sov/agents-workshop>

Was	Wofür
Prompts zum Kopieren.md	die Startkonfiguration, sauber zum Herauskopieren
data/CRM/	der CRM-Export als Excel-Datei, drei Tabellenblätter
data/Projektdokumente/	Abnahmeprotokolle, Projektdoku und Gesprächsnotizen als PDF
data/Kundenkommunikation/	Mailverläufe und Telefonnotizen als PDF

Der Ordner data ist Nachschlagematerial, keine Arbeitsanweisung. Er enthält genau die Unterlagen, auf die euer Agent später zugreift. Werft vorab einen Blick hinein: Ihr müsst später beurteilen können, ob eine Aussage des Agenten dort steht oder nicht. Die Dokument- und Vorgangsnummern in den Dateinamen sind dieselben, die der Agent in seinen Antworten nennt.

1.2 Öffnet SAP Build: <https://sovanta-joule-studio-dev.eu10.build.cloud.sap/lobby>

1.3 Meldet euch mit den ausgeteilten Zugangsdaten an. Joule Studio läuft nur in **Chrome** oder **Edge**.

1.4 Nach dem Login solltet ihr diesen Screen sehen:

[Screenshot einfügen: SAP Build Lobby]

2. Projekt und Agent anlegen

2.1 In der Lobby: **Create** und als Projekttyp **Joule Agent and Skill** wählen.

2.2 Vergebt einen eindeutigen Projektnamen, der euren Namen enthält, zum Beispiel Sales Support Agent Wolter. Jeder arbeitet in einem eigenen Projekt.

[Screenshot einfügen: Projektanlage]

2.3 Öffnet das Projekt und legt im Tab **Artifacts** über **Create** einen **AI Agent** an.

2.4 Tragt Name und Description ein:



- **Name:** Sales Support Agent
- **Description:**

Dieser Agent hilft bei allen Fragen rund um Kunden, Vertrieb, Produkte, Preise und Projekte der Norbach Antriebstechnik.

Die Description entscheidet, **wann** Joule diesen Agenten aufruft. Merkt euch das, wir kommen in Abschnitt 5 darauf zurück.

2.5 Wählt Create. Ihr landet im Agent-Editor.

[Screenshot einfügen: Agent-Editor, leer]

3. Agent konfigurieren

Der Agent-Editor hat drei Textfelder, die sein Verhalten bestimmen. Tragt jetzt genau die folgenden Werte ein. Sie stehen auch in Prompts zum Kopieren.md im Repo.

3.1 Expertise beschreibt die Rolle des Agenten. Tragt ein:

Du bist ein hilfreicher Assistent für den Vertrieb.

3.2 Instructions beschreibt, was der Agent tun soll. Tragt ein:

Beantworte die Fragen des Nutzers zu Kunden. Nutze dafür die verfügbaren Tools. Fasse die Ergebnisse verständlich zusammen.

3.3 Additional Context lasst ihr **leer**.

3.4 Model Settings lasst ihr zunächst auf den Standardwerten. Zur Orientierung, was die Felder bedeuten:

Feld	Bedeutung
Base Model	formuliert die Antwort an den Nutzer
Advanced Model	erstellt den Plan und entscheidet, welches Tool aufgerufen wird
Backup LLM	springt ein, wenn das primäre Modell zu langsam oder nicht erreichbar ist

3.5 Agent Execution Steps lasst ihr ebenfalls auf Standard:

Feld	Bedeutung
Maximum Number of Thinking Steps	wie viele Schritte der Agent maximal machen darf, bevor er abbricht
Pre-processing	zerlegt die Aufgabe und erstellt einen Plan, bevor gearbeitet wird
Post-processing	bündelt Zwischenergebnisse zu einer Endantwort

3.6 Agent Output: Format auf **Markdown** setzen.

4. Tools anbinden

Der Agent kann bisher nichts außer reden. Jetzt kommen vier Datenquellen dazu, bewusst je eine pro Datentyp.

4.1 MCP-Server: unstrukturierte Daten

Projektdokumentation, Abnahmeprotokolle, Gesprächsnotizen und Mailverläufe liegen hinter einem MCP-Server, der für euch bereits bereitsteht.

1. Wechselt im Agenten in den Tab **MCP Servers**



2. Wählt **Add MCP Server**

3. Name: workshop-docs

4. Description:

Durchsucht Projektdokumentationen, Abnahmeprotokolle, Gesprächsnotizen und Mailverläufe der Norbach Antriebstechnik.

5. Destination: Workshop_Docs_MCP aus der Liste wählen

6. Die Advanced Options (Path, Namespace, Timeout) bleiben auf Standard

7. Unter Tools seht ihr jetzt, was der Server anbietet. Es sollten drei Tools sein:

Tool	Was es tut
search_project_docs	sucht in Projektdokumenten, optional auf einen Kunden eingegrenzt
get_document	liefert den Volltext eines Dokuments zu einer Dokument-ID
search_communication	liefert Mailverläufe und Gesprächsnotizen zu einem Kunden, chronologisch

8. Add Server

[Screenshot einfügen: MCP Server hinzufügen, Tool-Liste]

4.2 Calculator: Rechenlogik

1. Unter Tools: Add Tool und dann **Calculator**

2. Name: Calculator

3. Description:

Führt mathematische Berechnungen aus.

4. Add

Die Description eines Tools ist der einzige Hinweis, den der Agent bekommt, wann er das Tool benutzen soll. Auch darauf kommen wir in Abschnitt 6 zurück.

4.3 Joule Skill 1: Kundenstammdaten

Strukturierte CRM-Daten kommen über Joule Skills. Die dahinterliegenden Actions sind bereits angelegt und in der Library veröffentlicht, ihr baut nur den Skill drumherum.

1. Unter Tools: Add Tool, dann **Joule Skill**, dann **Create Joule Skill**

2. Name: Get Customer Master Data

3. Description:

Returns master data for a customer of Norbach Antriebstechnik by customer name, including customer ID, industry, key account manager, year-to-date revenue and payment terms. Returns multiple results if several customers match the name.

Hinweis: Name und Description von Skills müsst ihr auf Englisch schreiben, auch wenn ihr später auf Deutsch mit dem Agenten chattet. Joule übersetzt die Nutzeranfrage im Hintergrund und matcht gegen die englische Beschreibung.

4. Create. Es öffnet sich ein neuer Browser-Tab mit dem Skill-Editor.

[Screenshot einfügen: Skill-Editor, leer]

5. Öffnet rechts das Seitenpanel und geht auf den Tab **Parameters**

6. Bei Skill Inputs auf **Configure**, dann **Add Input**:

- Name: customerName



- Description: Name of the customer to look up
- **Required** anhängen
- **Apply**

7. Bei **Skill Outputs** auf **Configure**, dann **Add Output** für jedes Feld:

Name	Description	List
customerId	Internal customer ID	nein
customerName	Full legal name of the customer	nein
industry	Industry of the customer	nein
keyAccountManager	Responsible key account manager	nein
revenueYtd	Year-to-date revenue in EUR	nein
paymentTerms	Agreed payment terms	nein

Apply

8. Fügt auf der Arbeitsfläche über das **Plus**-Symbol eine Aktivität vom Typ **Action** hinzu

9. **Browse All Actions**, sucht nach Workshop CRM und wählt die Action **Get Customers**

10. Im Seitenpanel, Tab **Input**: mappt den Skill-Input customerName auf den Input-Parameter der Action

11. Im Seitenpanel, Tab **General**: trägt als Destination-Variable Workshop_CRM_API ein

12. Mappt die Outputs der Action auf eure Skill-Outputs

13. Speichern und zurück in den Agent-Tab wechseln

[Screenshot einfügen: Action im Skill, Input-Mapping]

4.4 Joule Skill 2: Offene Opportunities

Gleiches Vorgehen, zweiter Skill.

1. **Add Tool, Joule Skill, Create Joule Skill**

2. **Name**: Get Open Opportunities

3. **Description**:

Returns all open sales opportunities for a given customer ID of Norbach Antriebstechnik, including title, value, currency, probability, sales phase and expected close date.

4. **Create**

5. **Skill Input**: customerId, Description Internal customer ID, **Required**

6. **Skill Outputs** als **List**:

Name	Description	List
opportunities	List of open opportunities	ja

Je Opportunity: opportunityId, title, value, currency, probability, phase, closeDate

7. **Action** hinzufügen, **Get Opportunities** aus Workshop CRM wählen

8. Input-Mapping und Destination-Variable wie bei Skill 1

9. Speichern

Beachtet: Skill 2 braucht eine customerId, die nur Skill 1 liefert. Der Agent muss die beiden also in der richtigen Reihenfolge aufrufen. Ob er das von allein tut, werdet ihr gleich sehen.

5. Testen und beobachten

Jetzt läuft der Agent zum ersten Mal. **In dieser Phase ändert ihr noch nichts.**



Das klingt unnötig streng. Wer aber sofort am Prompt schraubt, weiß am Ende nicht mehr, welche Änderung was bewirkt hat.

5.1 Wählt im Agent-Tab **Test**. Es öffnet sich eine Testumgebung mit drei Ansichten:

Ansicht	Was sie zeigt
Timeline	die Ereignisse des Agenten, verschachtelt und aufklappbar. Links chattet ihr
Editor	die Konfiguration des Agenten, während des Tests schreibgeschützt
Chart	wie viel Zeit jeder Schritt gebraucht hat

Über **Traces** seht ihr die Gedanken, Inputs und Outputs des Agenten. Über das **Debug**-Symbol öffnet ihr die **Request Logs** mit den tatsächlichen Aufrufen.

[Screenshot einfügen: Testmodus mit Timeline und Traces]

5.2 Stellt der Reihe nach diese vier Fragen:

#	Frage
T1	Bereite mir den Termin mit Halstrom Anlagenbau vor.
T2	Wie hoch ist unser gewichteter Pipeline-Wert bei Halstrom?
T3	Was weißt du über Kessler Systemtechnik?
T4	Welchen Preis können wir Halstrom zusagen?

Ein Durchlauf kann ein bis zwei Minuten dauern. Bei Fehlermeldungen hilft oft ein Reload.

5.3 Füllt für jede Frage diesen Bogen aus. Schaut dabei in die Timeline, nicht nur auf die Antwort:

Frage	Welche Tools wurden aufgerufen, in welcher Reihenfolge?	Was ist an der Antwort falsch, erfunden oder unvollständig?
T1		
T2		
T3		
T4		

5.4 Drei Hinweise, worauf ihr besonders achten solltet:

- Vergleicht die **Zahlen in der Antwort** mit dem, was in der Timeline tatsächlich an Daten zurückkam. Nehmt zur Not den Taschenrechner
- Schaut nach, welche Tools gar nicht aufgerufen wurden. Eine unvollständige Antwort sieht genauso aus wie eine vollständige
- Prüft in den Unterlagen im Repo nach, ob es das, was der Agent behauptet, überhaupt gibt

6. Debuggen und verbessern

Jetzt dürft ihr ändern. Aber gezielt: pro gefundenem Problem **eine** Änderung, danach erneut testen.

6.1 Entscheidet für jedes Problem zuerst, **wo** der Fix hingehört. Es gibt vier Orte, und die Wahl ist nicht beliebig:

Ort	Wofür er da ist
-----	-----------------



Instructions	Ablauf und Reihenfolge: was der Agent in welcher Situation tun soll
Expertise	Rolle, Zuständigkeit und Grenzen des Agenten
Tool-Description	wann genau ein bestimmtes Tool benutzt werden soll
Additional Context	Domänen- und Fachregeln, die unabhängig vom Ablauf gelten. Beginnt jeden Block mit ##

6.2 Leitfrage, wenn ihr feststeckt:

Woher hätte der Agent wissen sollen, dass er das anders machen soll?

6.3 Wenn ihr eine Änderung gemacht habt, stellt **dieselbe** Frage noch einmal und schaut in der Timeline nach, ob sich der Ablauf wirklich geändert hat. Eine bessere Antwort bedeutet nicht automatisch, dass der Agent den richtigen Weg genommen hat.

6.4 Typische Fragen, an denen ihr euch entlanghangeln könnt:

- Warum ruft der Agent den Calculator nicht auf, obwohl er rechnet?
- Warum fragt er bei zwei ähnlich heißen Kunden nicht nach, welcher gemeint ist?
- Warum sagt er nicht, dass er zu einem Kunden nichts gefunden hat?
- Warum kennzeichnet er eine 14 Monate alte Preisangabe nicht als unverbindlich?

7. Deployen und in Joule prüfen

Der Testmodus ist nicht dasselbe wie der Echtbetrieb. Zum Abschluss bringt ihr den Agenten in Joule.

7.1 Beendet den Test über **Exit Test**.

7.2 Released das Projekt.

7.3 Deployt das Projekt in eure Environment.

7.4 Öffnet Joule und stellt dort noch einmal die Frage T1.

[Screenshot einfügen: Agent in Joule]

7.5 Vergleicht: Verhält er sich genauso wie im Testmodus? Achtet besonders darauf, ob Joule die Antwort des Agenten umformuliert. Das steuert ihr im Agenten über **Agent Output** und die Option **Allow Joule to interpret the output of the agent**.

8. Zusatz-Challenges

Wenn ihr früher fertig seid, sucht euch etwas aus.

8.1 Description schärfen. Stellt eurem Agenten eine Frage, die nichts mit Vertrieb zu tun hat, zum Beispiel nach dem Urlaubsanspruch. Formuliert dann die Description so um, dass er gar nicht erst anspringt. Die Description steuert, wann Joule diesen Agenten überhaupt aufruft.

8.2 Modelle tauschen. Wechselt unter Model Settings das Advanced Model, also das Modell, das plant und Tools auswählt. Stellt dieselben Fragen erneut. Was ändert sich, was nicht?

8.3 Denkschritte begrenzen. Setzt Maximum Number of Thinking Steps deutlich herunter. Ab wann bricht der Agent ab, und wie merkt der Nutzer das?

8.4 Pre- und Post-processing abschalten. Schaltet Pre-processing aus und stellt T1 erneut. Danach dasselbe mit Post-processing. Die Doku sagt, gutes Prompting macht Pre-processing entbehrlich. Stimmt das für euren Agenten?

8.5 Human in the Loop. Aktiviert bei einem eurer Skills **Require user confirmation before running** und schreibt einen **Prompt for Generated Response**. Wann ist so eine Rückfrage sinnvoll, wann nur lästig?



8.6 Output erzwingen. Stellt das Agent Output Format auf JSON. Was gewinnt ihr, was verliert ihr?

8.7 Dritter Skill. Baut einen Skill Get Order History auf die Action **Get Orders** und lasst den Agenten den effektiven Rabatt gegen die hinterlegte Staffel prüfen.

8.8 Instructions weglassen. Löscht die Instructions komplett. Kommt der Agent allein zurecht? Was genau geht verloren?

9. Wenn etwas nicht funktioniert

Symptom	Wo ihr zuerst schaut
MCP-Server zeigt keine Tools	Destination im Dropdown richtig gewählt? Sonst kurz melden
Skill taucht beim Agenten nicht auf	Skill-Tab gespeichert? Agent-Tab neu laden
Action nicht in der Liste	In der Action-Suche Workshop CRM eingeben, nicht den Skill-Namen
Agent antwortet gar nicht	Seite neu laden. Danach Request Logs über das Debug-Symbol prüfen
Timeline bleibt leer	Test über Exit Test beenden und neu starten